

Dr. Isah Charles Saidu

Lead Instructor

Liatris Systems

AI & Technology Solutions

Session Duration: 1 Hour 30 Minutes

July 5, 2026



Liatris Systems



In partnership with the Independent National Electoral Commission

Session Roadmap

Part 1: Computer Vision

- ⚙ What is Computer Vision?
- 🖼 Image Processing Basics
- ⚙ Key Techniques

Part 2: OpenCV Fundamentals

- </> OpenCV Library
- ⚙ Image Operations
- ☰ Feature Detection

Part 3: OCR

- ⚙ What is OCR?
- 💻 Tesseract OCR
- ✓ Document Verification

Part 4: INEC Applications

- 🛡 EC8A Tamper Detection
- ✓ BVAS-IReV Monitor
- 🔑 Implementation

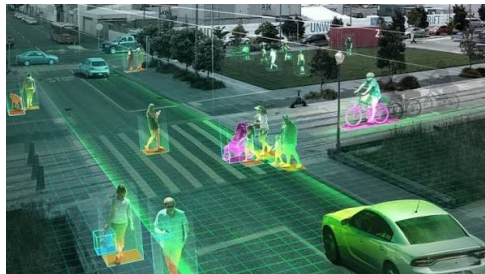
What is Computer Vision?

What is Computer Vision?

Computer Vision is a field of AI that enables computers to **interpret and understand** the visual world through images and videos.

Key Capabilities

- **Image Classification** – What is in the image?
- **Object Detection** – Where are objects?
- **Image Segmentation** – What belongs to what?
- **OCR** – What does the text say?
- **Feature Matching** – What is similar?



How Computer Vision Works

The Pipeline

1. **Image Acquisition** – Capture image
2. **Preprocessing** – Enhance quality
3. **Feature Extraction** – Identify patterns
4. **Analysis** – Interpret content
5. **Action** – Make decisions

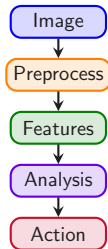


Image Processing Basics

Image Representation

Digital Images

- **Pixels** – Smallest unit
- **Grayscale** – Single channel (0-255)
- **RGB** – Three channels (Red, Green, Blue)
- **Higher Channels** – RGBA, HSV, etc.

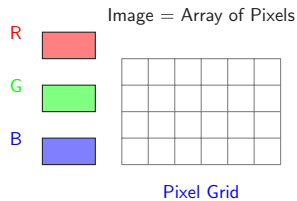
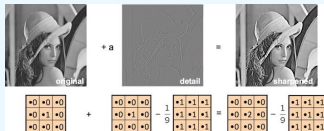


Image Processing Techniques

Filtering

- Blurring
- Sharpening
- Edge detection
- Noise reduction



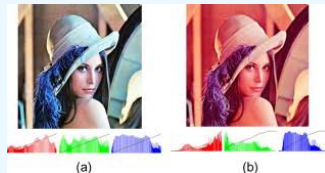
Geometric Transformations

- Resizing
- Rotation
- Cropping
- Perspective correction



Color Processing

- Brightness adjustment
- Contrast enhancement
- Histogram equalization
- Color space conversion



OpenCV Fundamentals

OpenCV Introduction

What is OpenCV?

- Open Source Computer Vision Library
- 2500+ optimized algorithms
- C++, Python, Java support
- Real-time computer vision
- Industry standard

Key Modules

- **core** – Basic data structures
- **imgproc** – Image processing
- **features2d** – Feature detection
- **objdetect** – Object detection
- **ml** – Machine learning
- **text** – OCR

OpenCV - Basic Operations

</> Code Example

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # 1. Read and display image
6 img = cv2.imread('election_form.jpg')
7 cv2.imshow('Original', img)
8 cv2.waitKey(0)
9
10 # 2. Convert to grayscale
11 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
12
13 # 3. Apply Gaussian blur
14 blurred = cv2.GaussianBlur(gray, (5, 5), 0)
15
16 # 4. Edge detection (Canny)
17 edges = cv2.Canny(blurred, 50, 150)
18
19 # 5. Thresholding
20 _, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

OpenCV - Basic Operations

</> Code Example

```
1  # 6. Resize image
2  resized = cv2.resize(img, (500, 400))
3
4  # 7. Rotate image
5  (h, w) = img.shape[:2]
6  center = (w // 2, h // 2)
7  matrix = cv2.getRotationMatrix2D(center, 45, 1.0)
8  rotated = cv2.warpAffine(img, matrix, (w, h))
9
10 # 8. Save result
11 cv2.imwrite('processed_image.jpg', edges)
```

OpenCV - Image Processing Pipeline

Typical Pipeline

1. Load image
2. Preprocess (grayscale, blur)
3. Edge detection
4. Contour detection
5. Feature extraction
6. Analysis

</> Pipeline Example

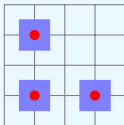
```
1 # Load image
2 img = cv2.imread('document.jpg')
3
4 # Preprocess
5 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
6 blur = cv2.GaussianBlur(gray, (5,5), 0)
7
8 # Edge detection
9 edges = cv2.Canny(blur, 50, 150)
10
11 # Find contours
12 contours, hierarchy = cv2.findContours(
13     edges, cv2.RETR_EXTERNAL, cv2.
14         CHAIN_APPROX_SIMPLE
15 )
16
17 # Draw contours
18 cv2.drawContours(img, contours, -1, (0,255,0), 2)
19 cv2.imshow('Contours', img)
```

Feature Detection

Feature Detection Techniques

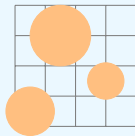
Corner Detection

- Harris Corner Detection
- Shi-Tomasi
- FAST



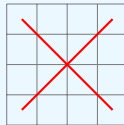
Blob Detection

- SIFT
- SURF
- ORB



Edge Detection

- Canny Edge Detector
- Sobel Operator
- Laplacian



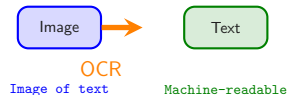
Introduction to OCR

What is OCR?

Optical Character Recognition (OCR) is the conversion of **images of text** into machine-readable text.

OCR Applications

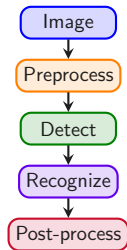
- Document digitization
- Form processing
- ID card verification
- Election form processing
- Handwritten text recognition



OCR Pipeline

OCR Process

1. **Image Acquisition** – Scan document
2. **Preprocessing** – Clean image
3. **Text Detection** – Locate text regions
4. **Character Recognition** – Identify characters
5. **Post-processing** – Correct errors



Tesseract OCR

Tesseract OCR

What is Tesseract?

- Open-source OCR engine
- Developed by HP (1985)
- Now maintained by Google
- Supports 100+ languages
- Python wrapper available

Key Features

- **Accuracy** – State-of-the-art
- **Languages** – 100+ languages
- **Formats** – Plain text, PDF, HOOCR
- **Training** – Customizable
- **Integration** – OpenCV, PIL

Tesseract - Code Example

</> Tesseract with Python

```
1 import cv2
2 import pytesseract
3 from PIL import Image
4
5 # Set Tesseract path (Windows)
6 pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'
7
8 # Method 1: Using OpenCV
9 img = cv2.imread('document.jpg')
10 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
11 text = pytesseract.image_to_string(gray)
12 print(text)
13
14 # Method 2: Using PIL
15 img = Image.open('document.jpg')
16 text = pytesseract.image_to_string(img)
17 print(text)
```

Tesseract - Code Example

</> Tesseract with Python

```
1 # Method 3: Extract with bounding boxes
2 data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)
3
4 # Get text and bounding boxes
5 for i, text in enumerate(data['text']):
6     if text.strip():
7         x = data['left'][i]
8         y = data['top'][i]
9         w = data['width'][i]
10        h = data['height'][i]
11        print(f"Text: {text}, Position: ({x}, {y})")
12
13 # Method 4: Preprocess for better accuracy
14 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
15 thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)[1]
16 text = pytesseract.image_to_string(thresh, config='--psm 6')
```

Tesseract Configuration

PSM Modes

- `--psm 0` – Orientation only
- `--psm 3` – Auto (default)
- `--psm 6` – Single uniform block
- `--psm 7` – Single text line
- `--psm 8` – Single word
- `--psm 11` – Sparse text

Useful Configurations

- `--oem 1` – LSTM only
- `--oem 3` – LSTM + Legacy (default)
- `--lang eng` – English
- `--lang fra` – French
- `--lang eng+fra` – Multiple languages
- `-c`
`tessedit_char_whitelist=0123456789`

INEC Applications

EC8A Tamper Detection

Problem Statement

- Detect physical tampering on EC8A forms
- Identify alterations and modifications
- Verify document authenticity

Solution Approach

- **Image Comparison** – Original vs scanned
- **Edge Detection** – Identify inconsistencies
- **Feature Matching** – Validate signatures
- **OCR** – Verify text integrity

</> Tamper Detection Code

```
1 import cv2
2 import numpy as np
3
4 # Load images
5 original = cv2.imread('ec8a_original.jpg')
6 scanned = cv2.imread('ec8a_scanned.jpg')
7
8 # Convert to grayscale
9 orig_gray = cv2.cvtColor(original, cv2.
    COLOR_BGR2GRAY)
10 scan_gray = cv2.cvtColor(scanned, cv2.
    COLOR_BGR2GRAY)
11
12 # Compute absolute difference
13 diff = cv2.absdiff(orig_gray, scan_gray)
```

EC8A Tamper Detection

Problem Statement

- Detect physical tampering on EC8A forms
- Identify alterations and modifications
- Verify document authenticity

Solution Approach

- **Image Comparison** – Original vs scanned
- **Edge Detection** – Identify inconsistencies
- **Feature Matching** – Validate signatures
- **OCR** – Verify text integrity

</> Tamper Detection Code

```
1 # Apply threshold
2 _, diff_thresh = cv2.threshold(diff, 30, 255, cv2.
    .THRESH_BINARY)
3
4 # Find contours of differences
5 contours, _ = cv2.findContours(
6     diff_thresh, cv2.RETR_EXTERNAL, cv2.
7         CHAIN_APPROX_SIMPLE
8 )
9
10 # Highlight tampered areas
11 for contour in contours:
12     if cv2.contourArea(contour) > 100:
13         x, y, w, h = cv2.boundingRect(contour)
14         cv2.rectangle(scanned, (x, y), (x+w, y+h),
15             (0, 0, 255), 2)
16
17 cv2.imshow('Tampered Areas', scanned)
```

BVAS-IReV Delay Monitor

Problem Statement

- Monitor BVAS-IReV result transmission
- Detect delays and anomalies
- Verify document timestamps

Solution Approach

- **OCR** – Extract timestamps
- **Image Analysis** – Verify document integrity
- **Anomaly Detection** – Identify delays
- **Real-time Monitoring** – Flag issues

</> Delay Detection Code

```
1 def check_delay(transmission_time, expected_time)
2     :
3     delay = (transmission_time - expected_time).
4         seconds
5     if delay > 300: # 5 minutes threshold
6         return True, delay
7     return False, delay
8
9 # Process images
10 for img in images:
11     timestamp = extract_timestamp(img)
12     if timestamp:
13         is_delayed, delay = check_delay(timestamp,
14                                         expected_time)
15         if is_delayed:
16             print(f"Delay detected: {delay}
17                   seconds")
```

BVAS-IReV Delay Monitor

Problem Statement

- Monitor BVAS-IReV result transmission
- Detect delays and anomalies
- Verify document timestamps

Solution Approach

- **OCR** – Extract timestamps
- **Image Analysis** – Verify document integrity
- **Anomaly Detection** – Identify delays
- **Real-time Monitoring** – Flag issues

</> Delay Detection Code

```
1 def check_delay(transmission_time, expected_time)
2     :
3     delay = (transmission_time - expected_time).
4         seconds
5     if delay > 300: # 5 minutes threshold
6         return True, delay
7     return False, delay
8
9 # Process images
10 for img in images:
11     timestamp = extract_timestamp(img)
12     if timestamp:
13         is_delayed, delay = check_delay(timestamp,
14                                         expected_time)
15         if is_delayed:
16             print(f"Delay detected: {delay}
17                   seconds")
```

Document Verification

Document Verification Pipeline

Verification Steps

1. **Scan Document** – High-quality image
2. **Preprocess** – Clean and enhance
3. **OCR Extraction** – Extract all text
4. **Validation** – Check against templates
5. **Verification** – Confirm authenticity

Key Checks

- **Text Matching** – Expected content present
- **Signature Verification** – Signature validation
- **Stamp Detection** – Official stamps present
- **Format Validation** – Proper template format
- **Anomaly Detection** – Unusual patterns

Best Practices

Computer Vision Best Practices

Image Quality

- Use high-resolution images
- Ensure good lighting
- Avoid blur and noise
- Use proper formats (PNG, JPG)

OCR Optimization

- Preprocess images
- Choose correct PSM mode
- Use language-specific models
- Post-process output

Remember: Garbage in, garbage out. Quality images = Quality results

Key Takeaways

Key Takeaways

What We Covered

1. **Computer Vision** – Understanding visual data
2. **OpenCV** – Image processing library
3. **OCR** – Converting images to text
4. **Tesseract** – OCR implementation
5. **INEC Applications** – EC8A tamper detection

💡 Key Principles

- ✓ **Preprocess** – Clean images first
- ✓ **Test** – Validate with diverse images
- ✓ **Optimize** – Tune parameters
- ✓ **Integrate** – Combine with ML
- ✓ **Monitor** – Track performance

Resources for Further Learning

Documentation

-  [OpenCV Documentation](#)
-  [Tesseract OCR Documentation](#)
-  [PyTesseract Documentation](#)

Tutorials

-  [OpenCV Python Tutorials](#)
-  [Tesseract OCR Tutorial](#)
-  [PylmageSearch](#)

Tools

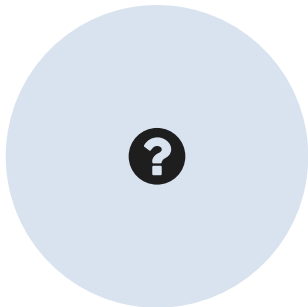
-  [OpenCV – Computer vision](#)
-  [Tesseract – OCR](#)
-  [PyTesseract – Python wrapper](#)
-  [PIL/Pillow – Image processing](#)

Communities

-  [OpenCV Forum](#)
-  [Tesseract GitHub](#)

Thank You

Questions & Discussion



"Computer vision is the eye of artificial intelligence."